

# NiosMess: изменения WebSocket сервера

## Документация для интеграции во Flutter-клиент

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| Серверный файл  | server_core/app/ws_manager.py                       |
| База данных     | Миграции не требуются                               |
| После деплоя    | Перезапустить FastAPI                               |
| Объём изменений | 6 изменений: 4 realtime-события + 2 новых WS action |

**Для фронтенда:** Главное изменение — больше не нужно делать refetch после read/edit/delete/reaction. Сервер сам рассылает realtime-события. Кроме этого, клиент должен отправлять decline\_call при отклонении входящего звонка и unregister\_fcm\_token перед logout.

## Кратко: что нужно внедрить на клиенте

- Добавить обработчики входящих событий: chat\_read, message\_edited, message\_deleted, message\_reaction.
- Обработать существующий end\_call как команду немедленно закрыть входящий баннер и завершить локальную call-сессию.
- При отклонении входящего звонка отправлять WS action decline\_call.
- Перед logout отправлять WS action unregister\_fcm\_token с FCM-токеном текущего устройства.
- Существующие new\_message / typing / new\_call / new\_ng\_post не менять.

## 1. Формат WebSocket обмена

Клиентские запросы по-прежнему передаются как action + payload. Сервер отвечает тем же action и возвращает payload/error; для request/response корреляции используется request\_id. Realtime broadcast приходит как отдельное событие с action и payload.

```
// Запрос клиента (схематично)
{
  "action": "<action>",
  "payload": { ... },
  "request_id": "<client request id>",
  "token": "<access token>"
}

// Обычный ответ сервера
{
  "action": "<same action>",
  "payload": { ... },
  "request_id": "<same request id>",
  "error": null
}

// Realtime broadcast
{
  "action": "message_edited | message_deleted | message_reaction | chat_read | end_call",
  "payload": { ... }
}
```

**Транспорт:** В ws\_manager уже есть шифрование WebSocket-пакетов/обмен ключами. Эта документация описывает логический action/payload после расшифровки; существующий транспортный слой менять не нужно.

## 2. Новый/изменённый контракт

| Направление      | action               | payload                                                       | Статус          | Что делает клиент                                      |
|------------------|----------------------|---------------------------------------------------------------|-----------------|--------------------------------------------------------|
| Client -> Server | mark_read            | {chat_id}                                                     | Уже существовал | Теперь иницирует chat_read для остальных участников    |
| Server -> Client | chat_read            | {chat_id, user_id}                                            | НОВОЕ событие   | Обновить read receipts без refetch                     |
| Client -> Server | edit_message         | {chat_id, message_id, content}                                | Уже существовал | После успеха сервер рассылает message_edited           |
| Server -> Client | message_edited       | serialized message                                            | НОВОЕ событие   | Заменить сообщение/preview; E2EE повторно расшифровать |
| Client -> Server | delete_message       | {chat_id, message_id}                                         | Уже существовал | После удаления сервер рассылает message_deleted        |
| Server -> Client | message_deleted      | {chat_id, message_id}                                         | НОВОЕ событие   | Удалить сообщение из state/cache                       |
| Client -> Server | react                | {chat_id, message_id, emoji}                                  | Уже существовал | Toggle своей реакции                                   |
| Server -> Client | message_reaction     | {chat_id, message_id, emoji, action}                          | НОВОЕ событие   | action = added   removed; изменить счётчик             |
| Client -> Server | decline_call         | {chat_id, room_id, message_id}                                | НОВЫЙ action    | Отклонить звонок и сразу завершить его у caller        |
| Server -> Client | end_call             | {chat_id, room_id, message_id, was_missed, duration, message} | Существовал     | Теперь обязательно закрывает call UI/сессию            |
| Client -> Server | unregister_fcm_token | {fcm_token}                                                   | НОВЫЙ action    | Удалить push token текущего устройства перед logout    |

## 3. Read receipts: chat\_read

Что изменено на сервере: после mark\_read сервер сбрасывает unread для текущего пользователя и рассылает остальным участникам чата событие chat\_read. Пользователь, который сам вызвал mark\_read, broadcast не получает.

```
// Клиент -> сервер
{
  "action": "mark_read",
  "payload": { "chat_id": 123 }
}

// Broadcast остальным участникам
{
  "action": "chat_read",
  "payload": {
    "chat_id": 123,
    "user_id": 42
  }
}
```

Рекомендуемая обработка на клиенте:

1. Найти чат payload.chat\_id.
2. Для сообщений, которые НЕ отправлены payload.user\_id, обновить локальный read-state согласно текущей модели клиента.
3. Обновить видимые галочки/read receipts без повторного запроса истории.
4. Не ждать FCM: служебное событие отправляется только по WebSocket (skip\_offline\_fcm=True).

## 4. Редактирование сообщений: message\_edited

После успешного edit\_message сервер сериализует обновлённый Message и рассылает message\_edited всем участникам чата. Автор также может получить broadcast (включая другие его соединения); поэтому обновление должно быть идемпотентным.

```
// Broadcast
{
  "action": "message_edited",
  "payload": {
    // полный serialized message в том же формате,
    // который использует сервер для сообщений
  }
}
```

- Upsert/replace сообщения по message.id, а не добавлять новый элемент.
- Если редактировалось последнее сообщение — обновить preview в списке чатов.
- Если сообщение E2EE — выполнить повторную клиентскую расшифровку нового содержимого.
- Сохранить обновлённую версию в локальный cache.

**Идемпотентность:** Не считайте response на собственный edit и broadcast двумя разными сообщениями. Оба должны сходиться в один message.id.

## 5. Удаление сообщений: message\_deleted

При delete\_message сервер удаляет реакции сообщения и рассылает message\_deleted. После этого удаляются media-файл (если был) и само сообщение.

```
{
  "action": "message_deleted",
  "payload": {
    "chat_id": 123,
    "message_id": 987
  }
}
```

- Удалить message\_id из state открытого чата.
- Удалить его из локального cache/database.
- Если это было последнее сообщение — пересчитать/обновить preview чата из локального состояния или существующего механизма синхронизации.
- Повторное message\_deleted для уже отсутствующего сообщения должно быть безопасным по-оп.

## 6. Реакции: message\_reaction

Существующий react работает как toggle реакции текущего пользователя. Если такая реакция уже есть — сервер удаляет её; иначе добавляет. Для других участников теперь отправляется message\_reaction. Инициатор broadcast не получает, потому что клиент уже применяет реакцию оптимистично.

```
// Добавление
{
  "action": "message_reaction",
  "payload": {
    "chat_id": 123,
    "message_id": 987,
    "emoji": "👍",
    "action": "added"
  }
}

// Удаление: payload.action = "removed"
```

- На своём устройстве оставить текущую optimistic update логику.
- На входящем message\_reaction: added => +1 / добавить реакцию; removed => -1 / удалить реакцию.

- Не ожидать echo broadcast для собственного react: сервер исключает user\_id инициатора.
- Не отправлять push для этого события: используется skip\_offline\_fcm=True.

## 7. Отклонение звонка: decline\_call

Добавлен новый WS action decline\_call. Он нужен, чтобы caller не ждал 60-секундный ringing timeout после того, как callee уже нажал «Отклонить». Сервер финализирует активный звонок как пропущенный и рассылает end\_call.

```
// Callee -> сервер
{
  "action": "decline_call",
  "payload": {
    "chat_id": 123,
    "room_id": "abc123...",
    "message_id": 987
  }
}

// Ответ на request
{
  "action": "decline_call",
  "payload": { "status": "declined" },
  "error": null
}
```

**Нюанс текущего backend:** message\_id присутствует в клиентском контракте, но обработчик decline\_call фактически использует room\_id. \_finish\_call достаёт chat\_id/message\_id из server-side active\_calls. Для совместимости контракт не упрощайте: отправляйте chat\_id + room\_id + message\_id.

При финализации сервер формирует end\_call примерно такого вида:

```
{
  "action": "end_call",
  "payload": {
    "chat_id": 123,
    "room_id": "abc123...",
    "message_id": 987,
    "was_missed": true,
    "duration": 0,
    "message": { /* serialized call-log message */ }
  }
}
```

Что клиент должен делать на end\_call:

1. Если room\_id соответствует входящему звонку — закрыть incoming-call banner/dialog.
2. Если room\_id соответствует активной call session — немедленно завершить локальную сессию (WebRTC/media/timers/route).
3. Обновить call-log message из payload.message, если клиент использует это поле.
4. Обработчик должен быть идемпотентным: повторный end\_call не должен повторно ломать навигацию/cleanup.

**Важно:** В \_finish\_call сервер также рассылает существующее событие edit\_message для call-log, а затем end\_call. Если старый клиент уже слушает edit\_message, не удаляйте этот обработчик из-за новых message\_edited событий.

## 8. Logout и FCM: unregister\_fcm\_token

Добавлен новый action для отзыва push-токена конкретного устройства. Сервер удаляет запись только если одновременно совпадают fcm\_token и текущий user.id.

```
// Перед logout
{
  "action": "unregister_fcm_token",
  "payload": {
    "fcm_token": "<current device token>"
  }
}

// Успех
{
  "action": "unregister_fcm_token",
  "payload": { "message": "FCM token removed" },
  "error": null
}
```

Рекомендуемый порядок logout:

1. Получить текущий FCM token устройства из существующего push-сервиса клиента.
2. Отправить unregister\_fcm\_token и дождаться response (если WebSocket ещё доступен).
3. После этого выполнить существующий logout / очистку access token и локальной сессии.
4. Если fcm\_token отсутствует, сервер вернёт error: "fcm\_token is required".

**Зачем:** Разлогиненное устройство перестаёт получать пуши пользователя. Удаляются не все токены аккаунта, а только переданный token текущего пользователя.

## 9. Пример роутера событий во Flutter/Dart

Ниже — пример структуры клиентской интеграции. Названия методов state/cache/callService нужно заменить на реальные методы проекта.

```
void onWsEvent(Map<String, dynamic> event) {
  final action = event['action'] as String?;
  final p = (event['payload'] as Map?)?.cast<String, dynamic>() ?? {};

  switch (action) {
    case 'chat_read':
      chats.markReadByUser(
        chatId: p['chat_id'],
        readerUserId: p['user_id'],
      );
      break;

    case 'message_edited':
      messages.upsertSerialized(p); // replace by message.id
      chats.refreshPreviewFromMessage(p);
      break;

    case 'message_deleted':
      messages.remove(p['chat_id'], p['message_id']);
      cache.removeMessage(p['message_id']);
      break;

    case 'message_reaction':
      messages.applyReactionDelta(
        chatId: p['chat_id'],
        messageId: p['message_id'],
        emoji: p['emoji'],
        added: p['action'] == 'added',
      );
      break;

    case 'end_call':
      callService.finishRemote(
        roomId: p['room_id'],
        wasMissed: p['was_missed'] == true,
        duration: p['duration'] ?? 0,
      );
  }
}
```

```
break;
}
}
```

## 10. Примеры отправки новых action

```
Future<void> declineIncomingCall(CallSession call) async {
  await ws.request('decline_call', {
    'chat_id': call.chatId,
    'room_id': call.roomId,
    'message_id': call.messageId,
  });
}

Future<void> unregisterPushBeforeLogout(String fcmToken) async {
  await ws.request('unregister_fcm_token', {
    'fcm_token': fcmToken,
  });
}
```

## 11. Realtime, offline и синхронизация

- chat\_read, message\_edited, message\_deleted и message\_reaction отправляются с skip\_offline\_fcm=True: эти служебные события не дублируются через FCM.
- Если клиент был offline, он должен догнать актуальное состояние через уже существующую загрузку истории/чатов при reconnect.
- new\_message / typing / new\_call / new\_pg\_post в рамках этих изменений не менялись.
- Для edit/delete/read/reaction realtime-путь должен обновлять локальный state без refetch в момент события.

## 12. Минимальные тест-кейсы для фронтенда

| Сценарий        | Действие                           | Ожидаемый результат                                                 |
|-----------------|------------------------------------|---------------------------------------------------------------------|
| Read receipt    | А читает чат                       | У В без refetch меняются read receipts по chat_read                 |
| Edit            | А редактирует сообщение            | У В текст меняется сразу; у А нет дубликата                         |
| Delete          | А удаляет сообщение                | У В сообщение исчезает из state/cache                               |
| Reaction add    | А ставит emoji                     | А видит optimistic update; В получает added                         |
| Reaction remove | А снимает emoji                    | А видит optimistic update; В получает removed                       |
| Decline call    | В отклоняет звонок А               | А получает end_call сразу, не ждёт 60 секунд                        |
| Logout push     | А выходит с device X               | Токен X удалён; другие токены А не удалены                          |
| Reconnect       | В был offline во время edit/delete | После reconnect состояние догоняется обычной историей/списком чатов |

## 13. Чеклист интеграции

- ☐ Добавлен case chat\_read.
- ☐ Добавлен case message\_edited с upsert по message.id.
- ☐ Добавлен case message\_deleted с удалением из state и cache.
- ☐ Добавлен case message\_reaction (added/removed).
- ☐ end\_call закрывает incoming UI и активную call session.
- ☐ Кнопка «Отклонить» отправляет decline\_call.
- ☐ Logout сначала отправляет unregister\_fcm\_token.
- ☐ Собственные реакции остаются optimistic (не ждать broadcast echo).

- Служебные события не завязаны на FCM.
- Проверены два клиента одновременно + reconnect/offline сценарий.

## 14. Что сервер не менял

- Миграции БД не требуются.
- E2EE media upload/download на сервере не менялись: сервер хранит/отдаёт байты как есть для is\_e2ee; шифрование остаётся клиентским.
- handle\_answer\_call отдельно в dispatcher не подключался и для текущего start\_call / join\_call / decline\_call / end\_call flow не требуется.

**После выката backend:** Перезапустить FastAPI, затем проверить контракт на двух устройствах/аккаунтах.